

Methods to solve Graph Coloring Problem

Repede Monica-Gabriela + Ursache Ana-Maria

April 8, 2026

-

1 Introduction

The Graph Coloring Problem (GCP) is a fundamental problem in graph theory and combinatorial optimization. Given an undirected graph $G = (V, E)$, where V denotes the set of vertices and E the set of edges, the objective is to assign a color to each vertex such that no two adjacent vertices share the same color. The minimum number of colors required to achieve such an assignment is known as the chromatic number of the graph and is denoted by $\chi(G)$.

Formally, a coloring is a function

$$c : V \rightarrow \{1, \dots, k\}$$

such that

$$c(u) \neq c(v), \quad \forall (u, v) \in E.$$

Determining whether a graph admits a valid coloring with at most k colors is an NP-complete decision problem, while minimizing the number of colors used is NP-hard. Due to its computational complexity, exact algorithms become impractical for large instances.

Proposed methods to solve GCP

2 TabuCol Method

2.1 What it is

- TabuCol is an application of **Tabu Search, a highly successful metaheuristic algorithm** specifically designed for the Graph Coloring Problem.
- The algorithm attempts to find a valid **k -coloring with 0 conflicts**, meaning no adjacent nodes share the same color.
- Unlike greedy approaches that color nodes incrementally, TabuCol starts with a **complete but random assignment of colors** (from 0 to $k-1$), which typically results in many edge conflicts.
- It then iteratively **repairs this invalid coloring** by selecting nodes involved in conflicts and changing their colors to minimize the total number of violations.
- To prevent cycling (flipping between the same two states), TabuCol uses a **short-term memory structure called the tabu list**. When a node changes from Color A to Color B, reverting to Color A is forbidden for a number of iterations.

2.2 Iterations

- **Core Loop Processing:** Each attempt to find a valid coloring with k colors runs for up to **50,000 iterations**. If a **0-conflict solution is found earlier, the process terminates immediately**.

- **Independent Restarts:** If no valid coloring is found within the iteration limit, the algorithm performs up to **3 independent random restarts** for the same value of k . If all attempts fail, k is considered infeasible.
- **Sequential Optimization Outer Loop:** The algorithm starts from a greedy upper bound on k and progressively **decreases** k , repeating the process until no further improvement is possible, thus approximating the chromatic number.

2.3 Optimizations Applied and Why

- **Dynamic Tabu Tenure Formula:**
 - **What:** The duration for which a move remains tabu is dynamically adjusted based on the number of colors k (typically $\approx 0.6 \times k$).
 - **Why:** A fixed tabu tenure often leads to poor performance, while a dynamic approach balances diversification and intensification.
- $\mathcal{O}(1)$ **Adjacency Color Lookup Map** (`adj_color_table`):
 - **What:** A matrix that stores, for each node, the number of neighbors assigned to each color.
 - **Why:** Instead of recomputing conflicts in $\mathcal{O}(\text{degree})$, this allows constant-time $\mathcal{O}(1)$ conflict evaluation, which is critical since the operation is executed millions of times.
- $\mathcal{O}(1)$ **Live Conflict Isolation Array:**
 - **What:** A dynamically maintained structure containing only nodes currently involved in conflicts.
 - **Why:** Enables direct random selection of conflicting nodes without scanning the entire graph, significantly improving efficiency.

3 Graph Neural Network (GNN) Method

3.1 What it is

- This method applies a **2-layer Graph Convolutional Network (GCN)** to the Graph Coloring Problem, leveraging self-supervised representation learning.
- Instead of directly predicting colors, the model **maps nodes into a low-dimensional embedding space**, where geometric proximity reflects coloring compatibility (close = same color, far = different colors).
- The approach is **fully self-supervised**, meaning no labeled colorings are required. Instead, a custom **hinge-based geometric loss** organizes the embedding space:
 - Adjacent nodes are pushed apart (distance $\geq m$).
 - Non-adjacent nodes are pulled closer (distance $\leq m/2$).
- After training, a **post-processing pipeline converts embeddings into discrete colors**, effectively transforming the combinatorial coloring problem into a geometric clustering task.

3.2 Model Architecture and Training

- **Feature Engineering and Preprocessing:**
 - Node features (X) are constructed using structural graph metrics:
 - * **Centrality measures:** degree, clustering coefficient, triangle counts, eigenvector centrality
 - * **Structural metrics:** betweenness centrality, core numbers

- Features are **normalized** (log-scaling) to ensure numerical stability.

- **GCN Architecture:**

- Layer 1: $H_1 = \text{ReLU}(\hat{A} \cdot X \cdot W_1)$
- Layer 2: $Z = \text{ReLU}(\hat{A} \cdot H_1 \cdot W_2)$
- $\hat{A}$ is the **normalized adjacency matrix** using the renormalization trick to prevent exploding/vanishing gradients:

$$\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$$

- **Weights are initialized using Xavier initialization.**

- **Self-Supervised Loss Function:**

The model does not use correct colors as labels. Instead, it uses a Hinge Loss function to organize the embedding space (Z):

- **Positive pairs (edges):**

$$\mathcal{L}_{\text{pos}} = \sum_{(i,j) \in E} \max(0, m - \|z_i - z_j\|)^2$$

- **Negative pairs (non-edges):**

$$\mathcal{L}_{\text{neg}} = \sum_{(i,j) \notin E} \max(0, \|z_i - z_j\| - m/2)^2$$

- **Total loss:**

$$\mathcal{L} = \mathcal{L}_{\text{pos}} + 0.3 \cdot \mathcal{L}_{\text{neg}}$$

- Gradients are computed and applied using **momentum-based SGD**.

3.3 Iterations

- **GNN Training (Epochs):** The number of training epochs is **dynamically scaled based on graph size**, preventing overfitting and unnecessary computation.
- **Iterated Local Search (ILS):** After initial coloring, a refinement phase runs for up to **500 iterations**, targeting the **rarest color class** and attempting to eliminate it by reassigning its nodes.

3.4 Embedding-Guided Coloring

Once the embeddings (Z) are trained, the code converts these high-dimensional vectors into discrete colors using a three-step process:

- **Priority-Based Assignment:**

- Nodes are ordered using a hybrid score: $score_i = 0.7 \cdot degree_i + 0.3 \cdot similarity_i$. Each node is assigned the color whose **embedding centroid is farthest** from its vector, promoting separation.

- **Iterated Local Search (ILS):**

- Attempts to reduce the number of colors by dissolving the smallest color class.

- **Conflict Repair:**

- A final pass ensures validity by resolving any remaining adjacent conflicts.

3.5 Optimizations Applied and Why

- **Vectorized Hinge Loss over Edges:**

- **What:** The backpropagation computes the hinge loss for every single edge simultaneously without looping through individual node pairs. It also samples negative random subsets (non-edges) identically.
- **Why:** Iterating node-to-node would choke the computation time. Vectorizing the cost over the adjacency matrix provides massive continuous chunks of arithmetic for the process scheduler.

4 Results

Table 1: Benchmark results: GNN vs. TabuCol (15 runs)

Instance	V	E	GNN				Tabu				Best k
			Mean k	Min	Max	Time(s)	Mean k	Min	Max	Time(s)	
Set 1: Myciel											
myciel3	11	20	4.00	4	4	0.0157	4.00	4	4	0.2864	4
myciel4	23	71	5.00	5	5	0.0243	5.00	5	5	0.4291	5
myciel5	47	236	6.00	6	6	0.0528	6.00	6	6	0.6091	6
myciel6	95	755	7.00	7	7	0.1830	7.00	7	7	0.9516	7
myciel7	191	2360	8.00	8	8	0.5788	8.00	8	8	1.4826	8
Set 2: Queen											
queen5_5	25	160	6.13	5	8	0.0384	5.33	5	6	0.3026	5
queen6_6	36	290	10.00	10	10	0.0601	7.73	7	8	0.7252	7
queen7_7	49	476	12.53	11	15	0.0890	8.00	8	8	0.8997	7
Set 3: Standard Instances											
anna	138	493	11.00	11	11	0.1299	11.00	11	11	1.4365	11
david	87	406	11.00	11	11	0.0918	11.00	11	11	1.2292	11
homer	561	1629	13.00	13	13	0.2577	13.00	13	13	1.1747	13
games120	120	638	9.13	9	10	0.1453	9.00	9	9	0.6182	9
Set 4: Medium											
flat300_28.0	300	21695	47.27	45	49	4.2284	35.80	35	36	8.7718	28
dsjc250.5	250	15668	42.13	41	44	2.2571	32.13	32	33	6.2456	28
le450_25c	450	17343	33.07	32	34	3.0228	29.00	29	29	3.9931	25
Set 5: Miles											
miles500	128	1170	20.07	20	21	0.2361	20.00	20	20	1.3744	20
miles1000	128	3216	44.60	44	45	0.5896	42.67	42	43	2.1177	42
miles1500	128	5198	73.00	73	73	0.9148	73.00	73	73	3.8054	73
Set 6: Large											
dsjc500.9	500	112437	177.47	174	181	13.0093	139.47	138	141	54.0298	126
dsjc1000.5	1000	249826	131.33	130	135	40.3504	102.67	101	104	63.5095	85

Table 2: Benchmark instances and performance statistics on ACO

Instance	V	E	Expected	Min	Max	Mean	Std. Dev.	Time (s)
myciel3	11	20	4	4	4	4	0	3,071
myciel4	23	71	5	5	5	5	0	5,574
myciel5	47	236	6	6	6	6	0	10,005
myciel6	95	755	7	7	7	7	0	12,731
myciel7	191	2360	8	8	8	8	0	25,533
queen5_5	25	160	5	5	5	5	0	3,522
queen6_6	36	290	7	7	7	7	0	4,232
queen7_7	49	476	7	7	7	7	0	6,038
anna	138	493	11	11	11	11	0	12,709
david	87	406	11	11	11	11	0	9,604
homer	561	1629	13	13	13	13	0	33,434
games120	120	638	9	9	9	9	0	5,528
miles500	128	1170	20	20	20	20	0	9,669
miles1000	128	3216	42	42	42	42	0	32,806
miles1500	128	5198	73	73	73	73	0	43,057